

Agent Orchestration

Sergei Telegin

github.com/TeleginS/agent-orchestration

157 tests passed. QA blocked the PR

TrafficRulesApp: my iOS app for the Spanish driving theory exam

32 new tests

Review approved
the change

QA found weaknesses
in the tests themselves

QA — Pass 1

Verdict:  BLOCKED on three in-scope findings.

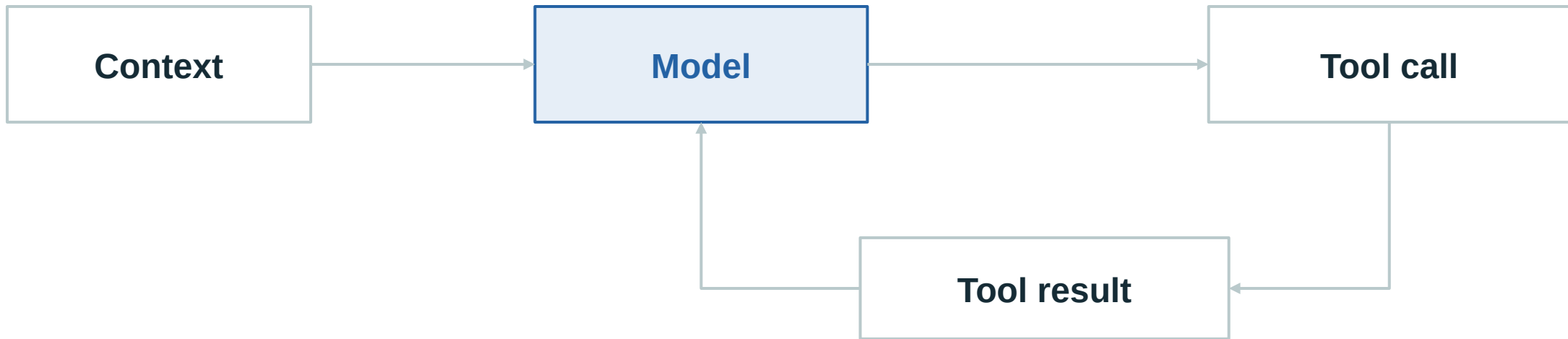
Test suite

Four runs, two destinations. Counts from `xcrun xcrresulttool get test-results summary`, not console grep.

#	Destination	Mode	Result
1	iPhone 17	default (parallel)	157 passed / 0 failed / 0 skipped
2	iPhone 11 Pro Max	default (parallel)	157 / 0 / 0
3	iPhone 17	<code>-parallel-testing-enabled NO</code>	157 / 0 / 0
4	iPhone 17	<code>-only-testing the two new classes</code>	32 / 0 / 0

Model, tools and execution environment

Stage result goes to the next role



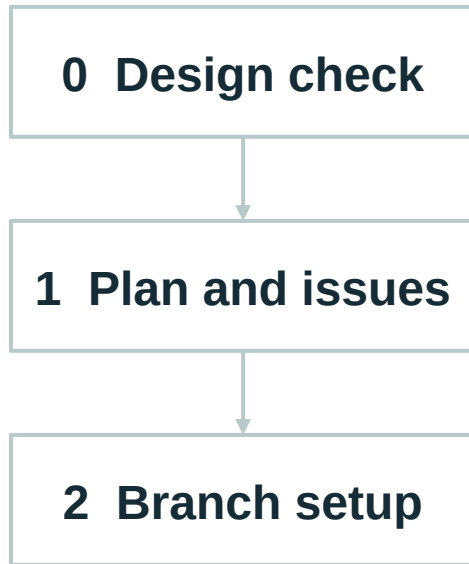
Harness

Runs tools, returns results and manages the execution loop

The complete delivery workflow

Current documented workflow

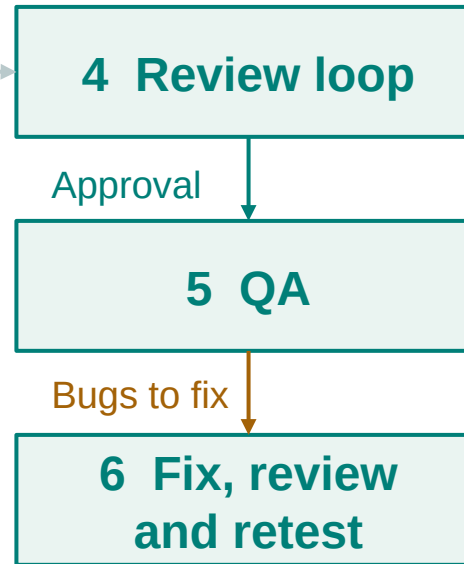
Preparation



Implementation

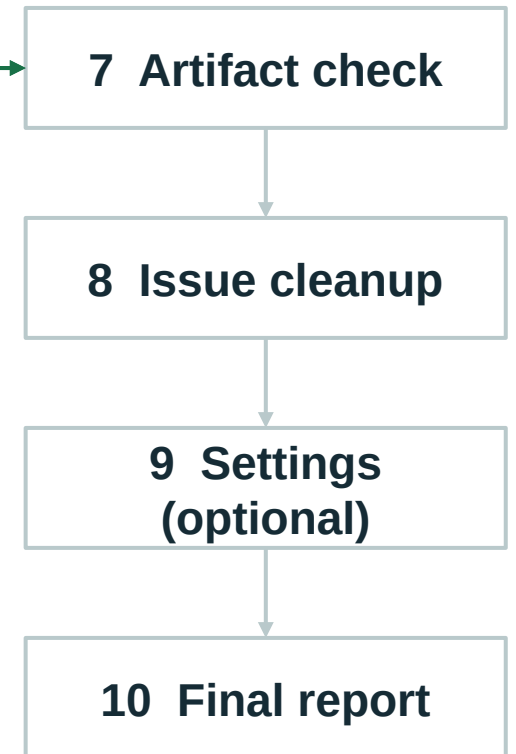


Verification



Both loops stop after
3 unsuccessful iterations

Completion



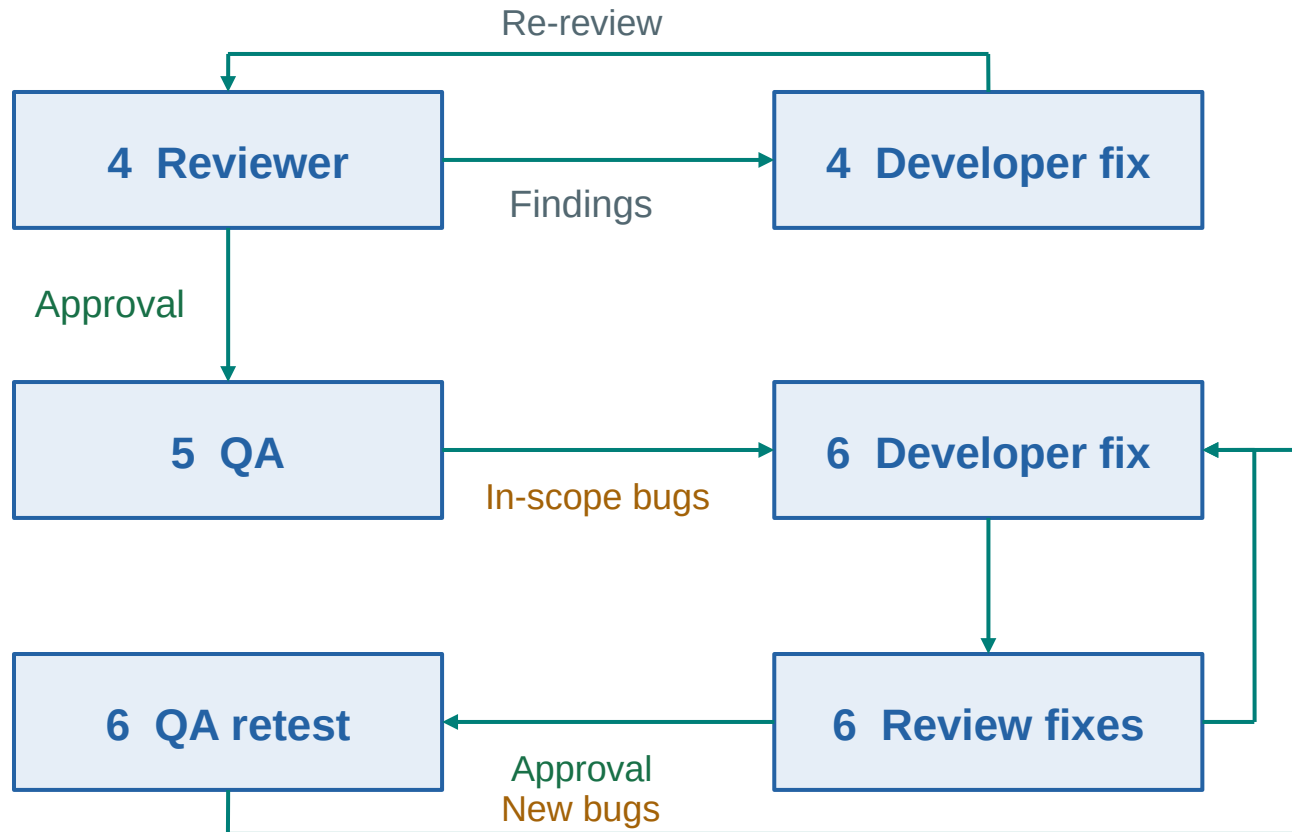
Orchestrator: routing and handoffs

Project profile: shared context

Review and QA loops

Current documented workflow

From step 3: PR and check results



Verified pre-existing bugs stay open. Unconfirmed origin remains blocking.

Rule 11

Review QA fixes before retesting

Limit: 3 iterations

QA green continues at step 7

A versioned change to the review route

July and August 2026

July: PR #143

Code Review — feature/cumulative-progress-statistics (PR #143)

Verdict:  APPROVED (posted as a comment — GitHub blocks self-approval via `gh pr review --approve` since I'm authenticated as the PR author: "Can not approve your own pull request")

[BUG] exitTest() double-counts session time into ProgressStore when it fires after natural completion #144 

26 August: rule 11

QA fix

Code review

QA retest

```
185 + 11. **Every code change after QA begins must pass code review before QA can be considered green.** QA bug fixes  
    go through ios-code-reviewer before qa-bug-tracker re-tests them.
```

Versioned instructions establish the change. Full historical session state is unknown.

PR #198: the route to QA

Recorded review and QA sequence

From step 3: PR and check results



Review 1: conditional approval

✓ **APPROVAL** (conditional — items 1 and 2 are worth one short follow-up pass; nothing blocks merge)

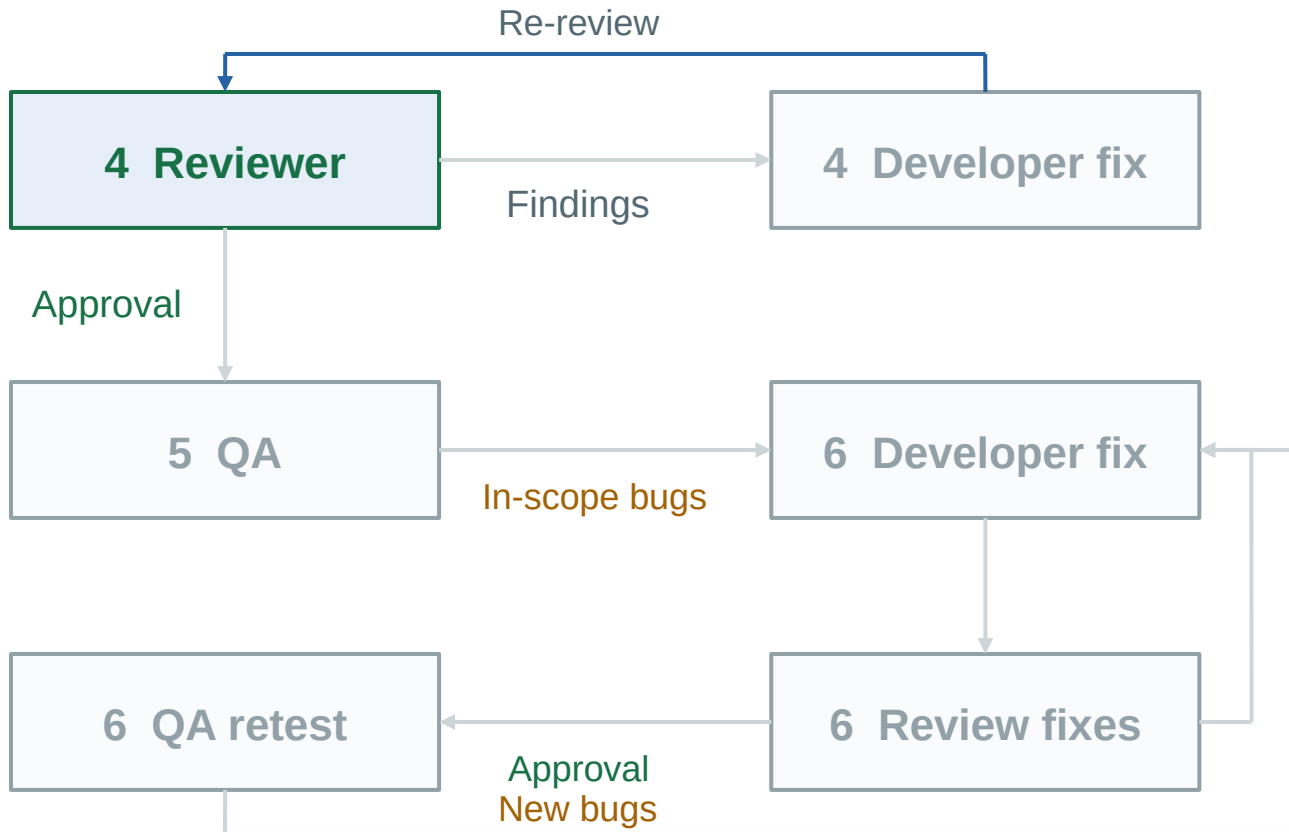
Quality findings prompt a developer follow-up

32 new tests
Application code unchanged

PR #198: the route to QA

Recorded review and QA sequence

From step 3: PR and check results



Review 2: approval

Code Review — Iteration 2

Re-review of the fix pass `ae659e9...e19273c` (+26 / -2).

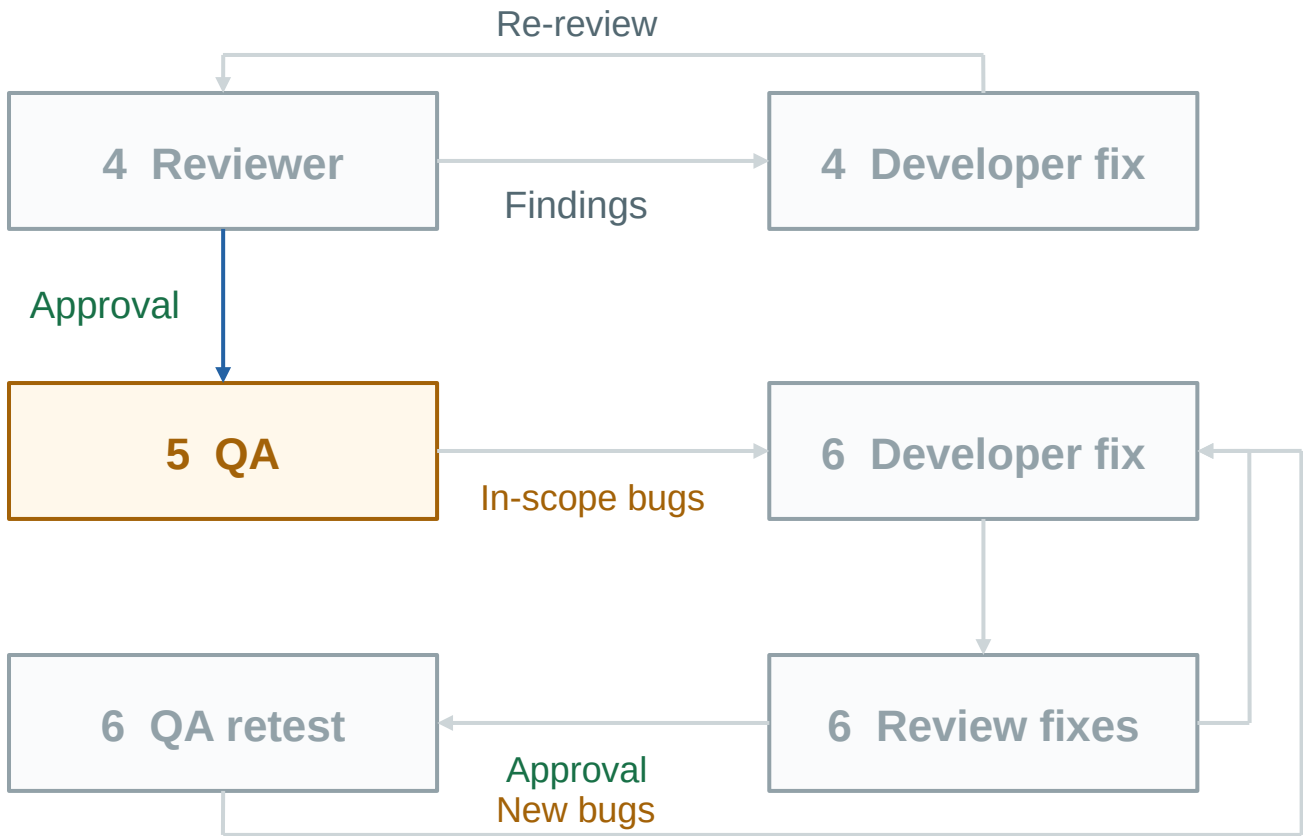
✅ APPROVAL

The same review stage checks the revised work

PR #198: the route to QA

Recorded review and QA sequence

From step 3: PR and check results



QA Pass 1: blocked 157 tests passed

QA — Pass 1

Verdict: 🛑 BLOCKED on three in-scope findings.

Test suite

Four runs, two destinations. Counts from `xcrun xcrresulttool get test-results summary`, not console grep.

#	Destination	Mode	Result
1	iPhone 17	default (parallel)	157 passed / 0 failed / 0 skipped
2	iPhone 11 Pro Max	default (parallel)	157 / 0 / 0
3	iPhone 17	<code>-parallel-testing-enabled NO</code>	157 / 0 / 0
4	iPhone 17	<code>-only-testing the two new classes</code>	32 / 0 / 0

The assertion and the starting state

QA findings in PR #198

Illustrative example

Before	500 seconds
After	500 seconds

after > 0 **PASS**

after > before **FAIL**

- [🕒 \[BUG\] Five terminal-path tests assert an absolute totalTestSeconds instead of a delta #199 \(medium\)](#) — five single-path tests assert `totalTestSeconds > 0` instead of a delta. Whenever the store is non-zero on entry the assertion is a tautology and cannot fail even if the accumulation line were deleted from the method under test. This is the half of PR [🔗 Cumulative all-time Progress statistics #143](#) invariant 1 the task exists to pin; the "not twice" half was written correctly with real snapshots, so the file is internally inconsistent. Not theoretical — it already happened in this task, and the suite still leaves `progress*` residue behind (🕒 [\[BUG\] TestSession tests leave progress* keys in UserDefaults.standard #202](#)).

Paired fix

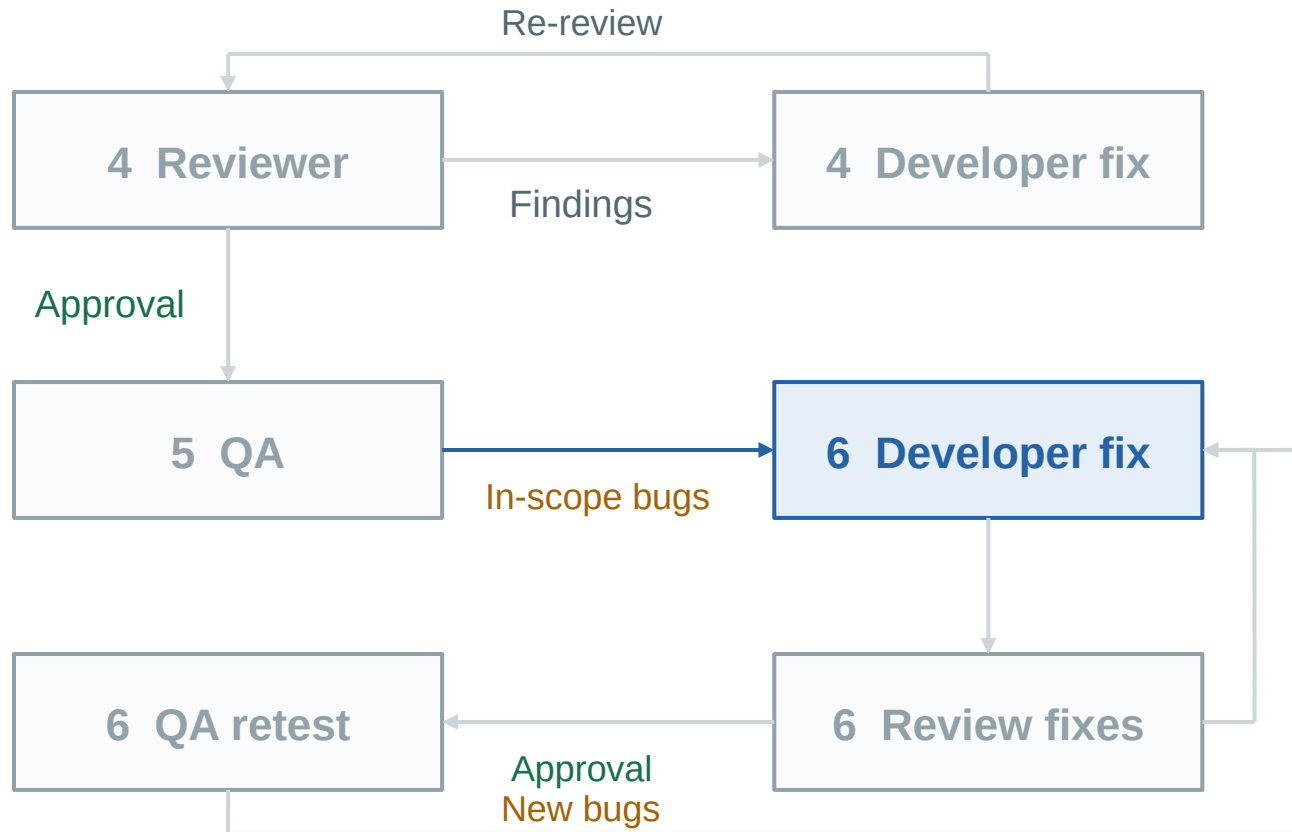
Compare with the prior value
Give each test its own store

The 500-second example illustrates a mechanism. It is not a measured result.

PR #198: fix, review and retest

Recorded review and QA sequence

From step 3: PR and check results



Developer: QA fixes

Commit **114cb54**

 **TeleginS** and **claude** committed 5 days ago · ✓ 3 / 3

Delta assertions
Isolated test storage

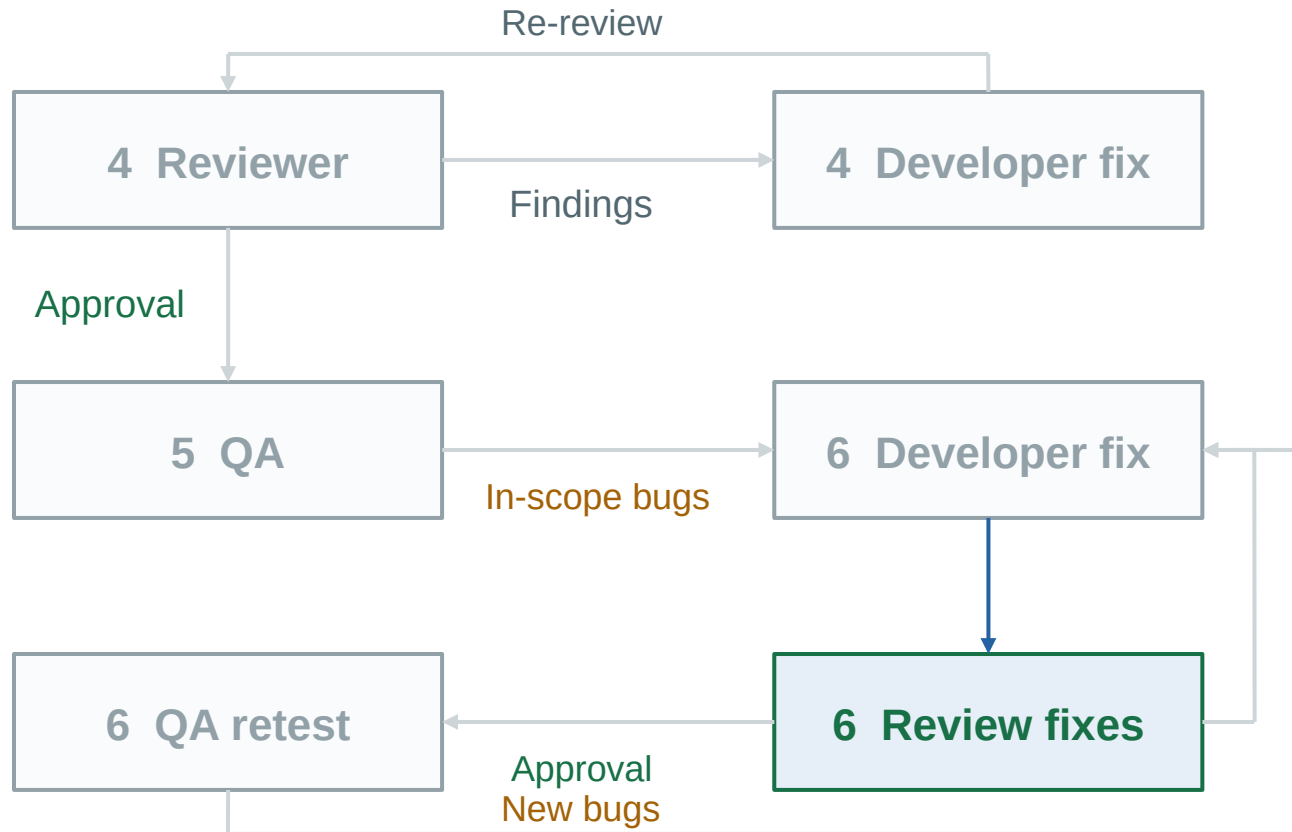
Same PR and branch

PR open at capture, 6 September 2026. QA green permits the completion steps.

PR #198: fix, review and retest

Recorded review and QA sequence

From step 3: PR and check results



Review 3: approval

Code Review — Iteration 3 (review gate on the QA fixes)

Pipeline rule 11: every code change made after QA begins must pass review before QA re-tests. Scope: `e19273c..114cb54` — 2 files, +49/-57, test code only.

✓ APPROVAL

After isolation, before = 0

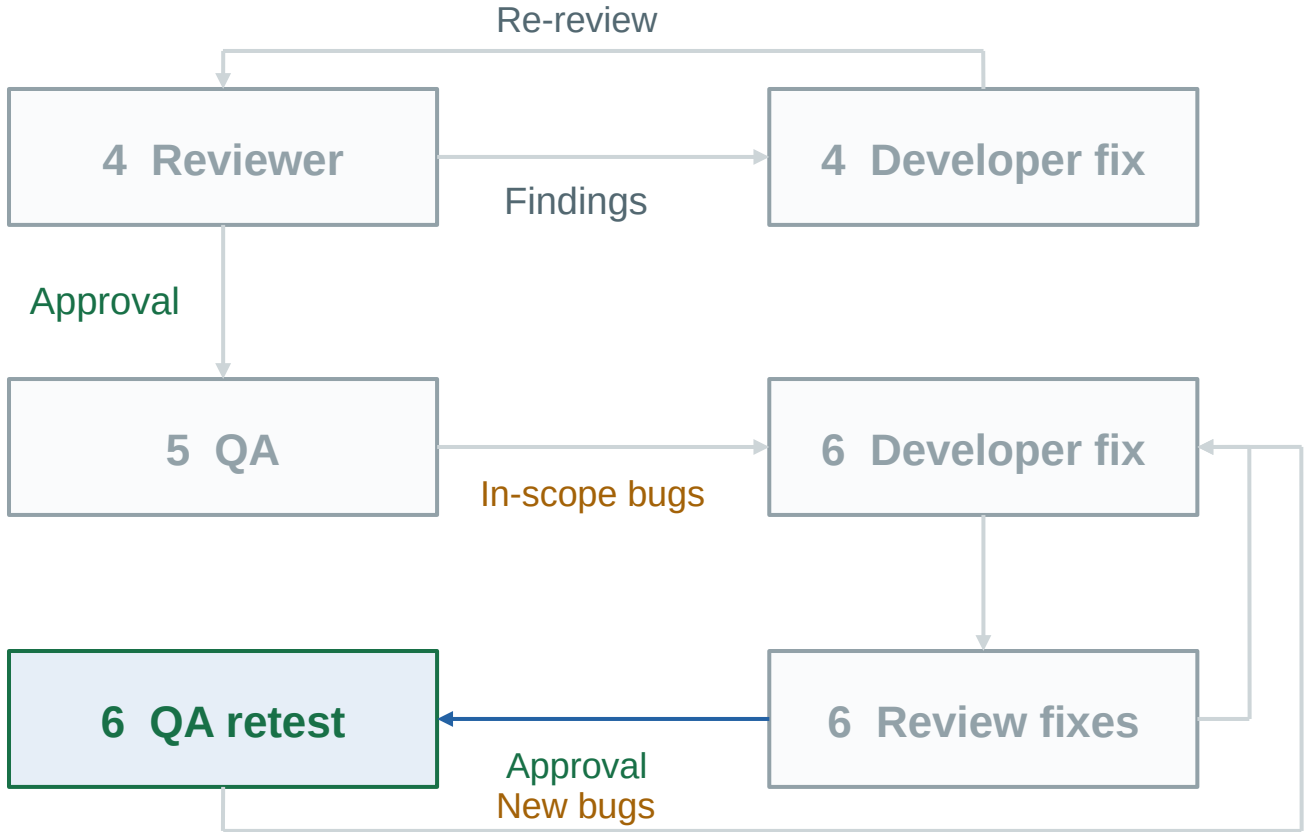
Both changes matter

PR open at capture, 6 September 2026. QA green permits the completion steps.

PR #198: fix, review and retest

Recorded review and QA sequence

From step 3: PR and check results



QA Pass 2: green

QA — Pass 2 (re-test)

Verdict: GREEN.

Run	Destination	Condition	Passed	Failed	Skipped
1	iPhone 17 (iOS 26.5)	baseline	157	0	0

Four full runs
157 passed in each

PR open at capture, 6 September 2026. QA green permits the completion steps.

The cost of this run

The same ProgressStore coverage task

Measure	Recorded result
Subagent launches	9
Elapsed time	About 1 h 31 min
Subagent tokens	1,097,667

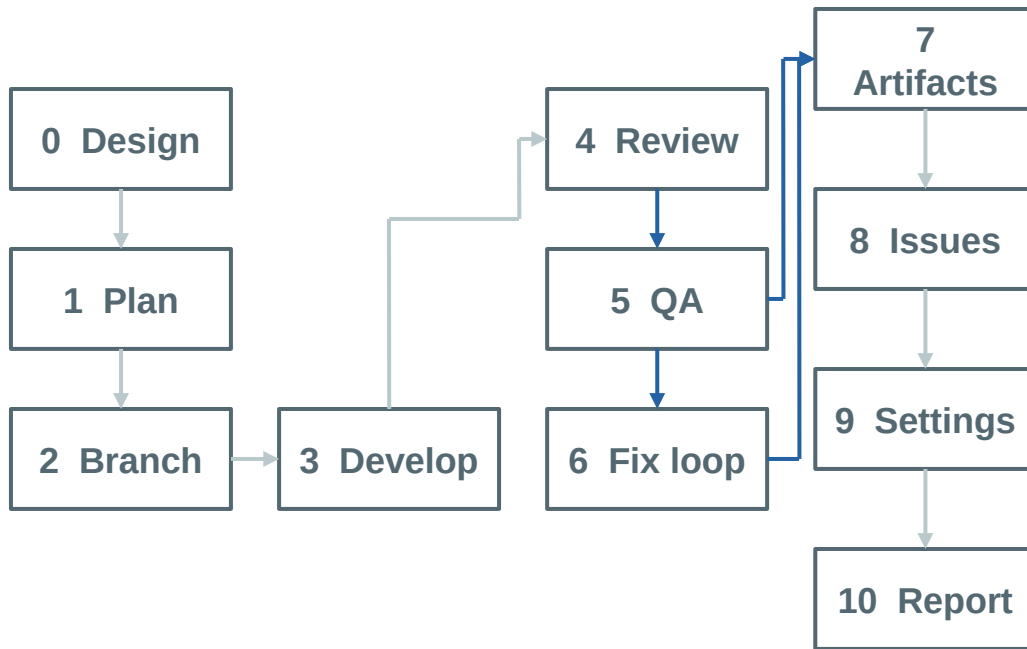
Planner 1 Developer 3
Reviewer 3 QA 2

Measured: subagent tokens

Step	Role	Tokens	Tool calls	Wall clock
1	task-planner	113,654	24	7m 10s
3	developer	159,422	59	24m 52s
4 · iter 1	code-reviewer	134,464	30	7m 57s
4 · fix	developer	74,767	19	4m 00s
4 · iter 2	code-reviewer	95,164	20	5m 41s
5	qa-tester	164,345	37	12m 31s
6a	developer	96,169	30	8m 32s
6b	code-reviewer	118,791	36	6m 43s
6c	qa-tester	140,891	47	13m 29s
Total	9 launches	1,097,667	302	1h 31m

Orchestrator tokens excluded. No matched single-agent baseline.

What I would measure next



**A checkable result
for each stage**

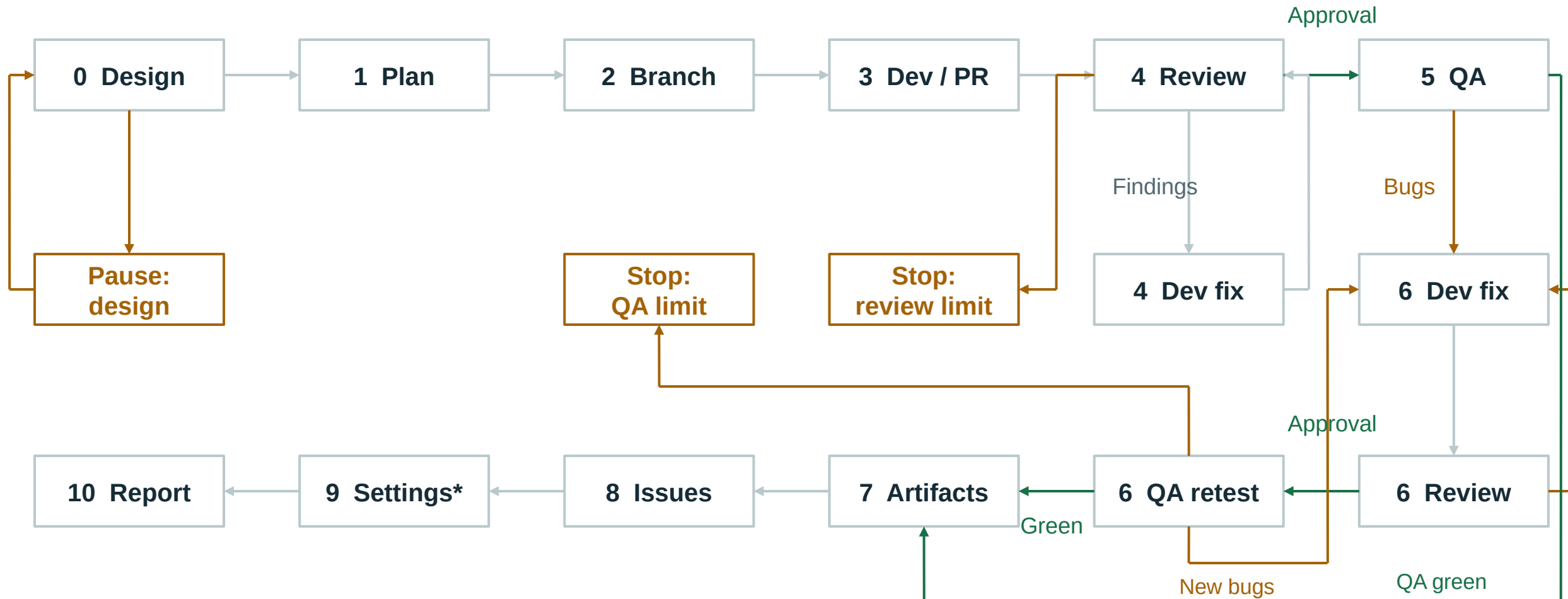
**A versioned record
of each run**

Next: validate approval and QA
against the code version

github.com/TeleginS/agent-orchestration

Full workflow and transition conditions

Reference for questions. Each repair loop stops after 3 unsuccessful iterations.



* Settings are optional. Verified pre-existing findings stay open. Unknown origin remains blocking. Completion includes artifact and issue checks. A green QA verdict does not establish merge.

The test fix in the actual diff

Assertions and isolated storage address connected concerns

```
@@ -53,46 +58,53 @@ final class TestSessionProgressTests: XCTestCase {
53 58 + func testCompleteTestAccumulatesTime() {
54 59     session.start(.quick(from: bank))
55 60     letSessionRun()
61 +     let before = progressStore.totalTestSeconds
56 62
57 63     session.completeTest()
58 64
59 -     XCTAssertGreaterThan(progressStore.totalTestSeconds, 0)
65 +     XCTAssertGreaterThan(progressStore.totalTestSeconds, before)
60 66 }
```

1. The  [BUG] Five terminal-path tests assert an absolute totalTestSeconds instead of a delta #199 delta form is correct, but credit it accurately. With the hermetic suite from  [BUG] New Progress tests are not isolated: UserDefaults.standard instead of a dedicated suite #200, before is 0 in all five tests — always — so > before is numerically identical to the old > 0. The

In the repaired fixture:

before = 0

after > 0

after > before

Numerically equivalent
for these five tests

The dirty-store example describes the original risk, not the repaired fixture.